

Homework 1: Due Tuesday, February 24, 2026 by 11:59pm

Please read these instructions to ensure you receive full credit on your homework. Submit the written portion of your homework as a **single PDF file through Courseworks** (less than 5MB). In addition to your PDF write-up, submit **all code** written by you in their original extensions through Courseworks (e.g., .m, .r, .py, etc.). Any coding language is acceptable, but your code should be your own. **Do not submit Jupyter** or other notebooks, but the original source code only. Do not wrap your files in .rar, .zip, .tar and do not submit your write-up in .doc or other file type. Your grade will be based on the contents of *one* PDF file and the original source code. Additional files will be ignored. **We will not run your code, so everything you are asked to show should be put in the PDF file. Show all work** for full credit.

Late submission policy: Late homeworks will have **0.1% deducted** from the final grade for **each minute late**. *Your homework submission time will be based on the time of your **last** submission to Courseworks.* Therefore, do not re-submit after midnight on the due date unless you are confident the new submission is significantly better to overcompensate for the points lost. You can resubmit as much as you like, but each time you resubmit be sure to upload **all** files you want graded! Submission time is non-negotiable and will be based on the time you submitted your last file to Courseworks. The number of points deducted will be rounded to the nearest integer.

Problem 1. (25 points)

In the first two problems, you will derive and implement an EM algorithm for the object recommendation problem discussed in class. Here, we have a data set of the form $\mathcal{R} = \{r_{ij}\}$ restricted to a subset of pairs $(i, j) \in \Omega$, where i can range from $1, \dots, N$ and j can range from $1, \dots, M$ and we let $r_{ij} \in \{\pm 1\}$. The goal of matrix factorization is to find a low rank approximation of this data.

To this end, let the unknown model variables be $u_i \in \mathbb{R}^d$ and $v_j \in \mathbb{R}^d$ for $i = 1, \dots, N$ and $j = 1, \dots, M$. Let $U = \{u_i\}$ and $V = \{v_j\}$. The model and priors are,

$$r_{ij}|U, V \stackrel{\text{iid}}{\sim} \text{Bernoulli}(\Phi(u_i^T v_j / \sigma)), \text{ for all } (i, j) \in \Omega, \quad u_i \stackrel{\text{iid}}{\sim} N(0, cI), \quad v_j \stackrel{\text{iid}}{\sim} N(0, cI).$$

The function $\Phi(\cdot)$ is the CDF of a standard normal random variable, as discussed in class. (Hint: You will find the probit regression discussion useful for this homework.)

The goal of this homework will be to derive and implement an EM algorithm for maximizing

$$\ln p(\mathcal{R}, U, V) = \underbrace{\int q(\phi) \ln \frac{p(\mathcal{R}, U, V, \phi)}{q(\phi)} d\phi}_{\mathcal{L}(U, V)} + \int q(\phi) \ln \frac{q(\phi)}{p(\phi|\mathcal{R}, U, V)} d\phi,$$

where the auxiliary variable ϕ is used in the probit setup similar to that discussed in class for probit regression: $\phi = \{\phi_{ij}\}$ for $(i, j) \in \Omega$ and $r_{ij} = \text{sign}(\phi_{ij})$, $\phi_{ij} \sim \text{Normal}(u_i^T v_j, \sigma^2)$.¹

¹Note: In the data, $r_{ij} \in \{-1, +1\}$ instead of $\{0, 1\}$. This won't impact the final derivation.

Problem 2 will focus on implementing the algorithm. To this end:

- a) Derive $q(\phi)$ for the EM algorithm. Hint: You will be able to show $q(\phi) = \prod_{(i,j) \in \Omega} q(\phi_{ij})$.
- b) Derive $\mathcal{L}(U, V)$. You can use the $\mathbb{E}_q[\cdot]$ notation and write the relevant expectation separately.
- c) Derive the U, V that optimize $\mathcal{L}(U, V)$. Hint: Derive for one (and therefore all) u_i and v_j .
- d) Summarize the final EM algorithm in a list of steps that are easy to read, yet detailed enough for someone to be able to implement it. (Be sure to write out $\ln p(\mathcal{R}, U, V)$.)

Problem 2. (25 points)

In this problem, you will implement the EM algorithm you derived in Problem 1 for finding a local optimal solution to $\ln p(\mathcal{R}, U, V)$. The data is provided on Courseworks along with a README file explaining the data. For your implementation, set $d = 5$, $c = 1$ and $\sigma^2 = 1$. Treat the users as U and the movies as V when implementing the algorithm.²

You will need to initialize your algorithm in some way. One possibility is to start by generating each u_i and v_j from $\text{Normal}(0, 0.1I)$. Please use this method for this assignment.

- a) Run your algorithm for 100 iterations and plot $\ln p(\mathcal{R}, U, V)$ for iterations 2 through 100.
- b) Rerun your algorithm for 100 iterations using 5 different random starting points. Plot the 5 different objective functions for iterations 20 through 100. Note: This is simply a repeat of Problem 2(a), only showing 5 objective functions instead of one and changing the x-axis.
- c) Predict the values given in the test set and show your results in a confusion matrix. Show the raw counts in this confusion matrix.

Problem 3. (25 points)

In this problem, you will implement the MCMC sampling approach discussed in Lecture 3 on the data from Problem 2. In this scenario, treat r_{ij} as a real-valued observation as described in the notes. Therefore, ϕ_{ij} does not appear in this problem. This is not ideal, but can still produce useful results. Again set $d = 5$, $c = 1$ and $\sigma^2 = 1$. Initialize U and V to all zeros. (It's worth thinking why this initialization wouldn't work for EM, but works for MCMC.) Since the algorithm was already derived in the notes, you don't need to rederive the Gibbs sampler.

- a) Write out the log joint likelihood for the model (for any d , c and σ^2), showing all steps in how you arrived at the final mathematical equation.
- b) Run your code for 1000 iterations. In two separate figures, plot the log joint likelihood for all iterations, and the log joint likelihood for iterations 100 to 1000.
- c) Using samples collected after every 25 iterations (starting with iteration 100), calculate a Monte Carlo approximation to $\mathbb{E}[r_{ij}|R]$ for each value in the test set. Predict +1 if this expectation is positive, and -1 otherwise. Show your test results in a confusion matrix as you showed them in Problem 2(c).

²Hint: Depending how you implement it, your code may be very fast or very slow. You may want to change the data format so you aren't looping over the list of ratings. Also, the EM algorithm will work if you change around the orders: e.g., update $q(\phi)$, then update U , then update $q(\phi)$, then update V . We didn't present EM this way—we presented it as doing "E" for everything then "M" for everything in one iteration—but it may help develop your understanding of EM if you think through why this suggestion also works. Your code will work if you ignore this hint, but it may end up being slow, and this hint is probably not the only way to faster code.